

Cryptographic Migration with Implicit Dependencies

Anonymous Submission*

* No Institution given

No contact details given

Abstract—Since the announcement of the PQC NIST standards, various standardization bodies and industry stakeholders have begun migrating protocols and security-related applications to meet recently announced national and international regulatory PQC migration requirements. Although replacing individual primitives with existing PQC building blocks appears feasible, migrating complex, heterogeneous systems remains challenging due to the multitude of dependencies between individual components as well as security and cryptographic assets. In this paper, we discuss different types of dependencies, propose a formal framework, and a methodology for identifying dependencies between the various components of a complex system, thereby supporting the migration process towards the use of PQC algorithms. The proposed methodology utilizes a dependency graph model to identify implicit and non-obvious dependencies between cryptographic assets, such as cryptographic primitives, protocols, and keys. To illustrate the methodology and demonstrate its potential, we formulate and analyze a proof-of-concept algorithm that identifies implicit dependencies and supports a clustering-based migration strategy, leveraging dependency-specific test functions and a migration oracle to validate the correctness of the migrated system resp. application.

Index Terms—Post-quantum cryptography, cryptographic migration, dependency graph

1. Introduction

With the advancement of quantum computers and regarding the growing amount of physical qubits, it has become crucial to consider the transition towards the usage of post-quantum cryptography (PQC), since the algorithms of Shor [28] and Grover [13] pose a significant threat to classical asymmetric cryptographic algorithms with the existence of a sufficiently scaled quantum computer [21]. One aspect of the challenge is the development of secure PQC algorithms; the migration of existing systems is also necessary. Migration is essential for maintaining long-term security; however, it necessitates meticulous planning, substantial resources, and collaboration among numerous stakeholders [15]. Several research groups and agencies have come up with different migration plans that range in how abstract they are and what they focus on. Some offer guidance at the governance level [4], while others deal with technical issues like how to implement hybrid constructions and which algorithms to replace [3, 5, 9].

In this work, we focus on the migration steps of identifying dependencies and proposing migration plans. As

the authors of [14, 16] have discussed, clustering mutually dependent nodes is a critical element in the development of a migration strategy, which is a sequence of migration steps that consider the underlying dependencies based on a graph-based model. As the migration process to PQC necessitates structural modifications to the system architecture, it is not limited to the replacement of algorithms within individual components. It also requires a detailed understanding of cryptographic dependencies between the components. Because of these dependencies, there are technical, organizational, and legal limits that need to be found and built into the migration plan. To resolve these components, we propose a semi-formal framework and a graph-based model. In this model, nodes represent cryptographic assets and system components, while edges represent either explicit or implicit dependencies. Explicit dependencies are apparent and primarily defined by structural architectural topologies, whereas implicit dependencies represent non-obvious dependencies that are based on domain knowledge or non-functional dependencies, such as security-based dependencies.

In addition, we examine and discuss different kinds of explicit and implicit dependencies that might occur during PQC migration. We examine the significance of these dependencies and suggest a methodology for identifying and managing them, informed by the CBOM concept from the CycloneDX project [22] and the model in [16]. More specifically, we incorporate a clustering-based graph model to establish a structured perspective on cryptographic elements and their explicit dependencies. After adding implicit dependencies between nodes, which are often missed in traditional methods, we improve this model. With an automotive case study, we demonstrate that dependencies significantly affect the clustering process. We show that cryptographic keys must be considered alongside because of their impact on components or services. Finally, we develop and evaluate a clustering and detection algorithm as a proof of concept for the proposed semi-formal framework.

The rest of the paper is structured as follows. Section 2 examines current cryptographic migration strategies, emphasizing dependency graph frameworks, asset identification techniques, and semi-formal models. To demonstrate various dependency classes in a practical context, the paper uses a representative automotive use case, which is introduced in Section 3. Section 4 revisits the fundamental migration-graph concepts and enhances them to incorporate cryptographic assets as nodes. In Section 5, an explicit refinement process and a taxonomy of dependency types, such as implicit and non-obvious dependencies,

are added to this model. Section 6 formalizes migration strategies using dependency-specific test functions and an oracle. In this section, we also present a clustering and detection algorithm. In the last section, we conclude and outline future work, while the appendices provide formal proofs and a machine-readable representation of the application example.

2. Comparison of Related Migration Models for Cryptography

As mentioned in the introduction, the transition to post-quantum cryptography (PQC) is now a major strategic issue for the security of information systems, since the development of PQC algorithms is only part of the solution. A large-scale migration of existing systems to use these new algorithms is essential to maintain long-term security. Since 2016, this issue has been institutionalized through the efforts of the National Institute of Standards and Technology (NIST), which initiated an international process to select the first quantum resistant algorithms [2]. This process, now mature, has led to the standardization of several key algorithms such as ML-KEM for encryption [23], and ML-DSA [24] and SLH-DSA[25] for signatures. At the same time, many national agencies, such as the NSA in the United States, the Federal Office for Information Security (BSI), French Cybersecurity Agency (ANSSI), Netherlands Organization for Applied Scientific Research (NTO), and European Telecommunications Standards Institute (ETSI), have published official guidelines incorporating these new algorithms into their long term security profiles. The BSI, for example, has updated its cryptographic recommendations since 2020 in the German national standard document for cryptographic primitives to include post quantum primitives [12] and updated it yearly thereafter.

2.1. Semi-Formal Models for Migration

Cryptographic migration of a system is not only achieved by a straightforward replacement of the algorithms in the source code; it is far more complex. It involves a profound transformation of chains of trust, data formats, key sizes, hardware capabilities and, above all (what concerns us) dependencies between components, especially in the PQC context. Therefore, models are required to support the migration of cryptographic primitives and protocols in systems. Current research aims at developing those models and methodologies to support the migration process to enable practitioners to transition from the use of classical symmetric cryptographic primitives to the use of PQC.

One of the most recent and promising approaches to semi-automated cryptographic migration planning is based on the notion of a structured inventory of cryptographic assets, as formalized via CBOMs proposed as part of the OWASP CycloneDX project [22]. The CBOM aims to provide a comprehensive and interoperable description of a system's cryptographic elements: algorithms, keys, certificates, hardware modules (such as smart cards or HSMs), libraries, and associated protocols. This inventory clarifies the implementation and usage relationships between these elements (we will call this type of relationship

functional dependencies). For example, a certificate can be linked to a public key, itself generated by a specific algorithm implemented in a cryptographic library used by an application.

This descriptive approach is supported by several recommendations from national cybersecurity agencies, such as ANSSI in France, the BSI in Germany, or TNO in the Netherlands, which advocate the creation of a cryptographic inventory as an essential prerequisite for any migration strategy to post-quantum algorithms [4, 5, 9]. The CBOM provides a standardized and automatable method for maintaining this inventory in complex and evolving systems.

2.2. Models for Asset Identification

Following [21], the initial step in the migration process is to identify the components that require consideration. The identification of the relevant (cryptographic) assets is the most relevant activity in the step of identification or diagnosis of the four generic migration process steps: (1) Identification, (2) Plan, (3) Execution, and (4) Maintenance¹. Various models have been proposed in the literature, ranging from purely policy-driven approaches to risk methodology-inspired approaches and the mentioned semiformal models. In this section, we provide a background to the existing methodologies in the literature as a basis for the subsequent discussion.

2.2.1. Policies-driven Approaches. National cybersecurity authorities have published official recommendations to support the migration to post-quantum cryptography. These policy-driven approaches are mainly based on the concept of security strength and provide guidelines for industry and government agencies. The French cybersecurity agency (ANSSI) follows a policy-driven approach. This approach [5] emphasizes the principle of caution. The agency recommends a gradual transition, based on cryptography and the use of hybrid architectures combining proven classical algorithms (e.g. RSA, ECC) and post-quantum candidates currently being standardized by NIST. ANSSI insists on maintaining security levels equivalent to those defined by its current standards (e.g. 128-bit security for common uses and 192-bit for sensitive environments). Therefore, all relevant cryptographic primitives and protocols that use a security level below 128-bit security level are considered assets relevant for migration.

The BSI follows a similar approach based on their security philosophy. In [9], the presented approach is based on the concept of the level of protection required depending on the sensitivity of the data and systems concerned. The BSI first recommends a preliminary assessment of existing systems to identify the cryptographic mechanisms currently in use and their potential exposure to quantum threats, and handles them as critical assets for the migration process. On this basis, it advocates a gradual transition via hybrid solutions (exploiting a combiner construction to ensure that a classical and post-quantum scheme is used, e.g., X-Wing [7]), particularly

1. Sometimes these steps are given alternative names but always aim at the core operation of identification, prioritization, verification, and execution.

for widely used communication protocols such as TLS and IKEv2, as well as for certificate management infrastructures. Finally, it recommends increasing the key sizes of symmetric algorithms (e.g. from AES-128 to AES-256) to compensate for the impact of Grover's algorithm, which reduces the effectiveness of symmetric keys in the presence of a relevant quantum computer. Therefore, all primitives and protocols with a security level at 128 or below 128-bit security (classical security model) are considered relevant assets for the transition.

The approach of the organization of TNO is also policy-driven, but more proactive and practical, domain-related compared with the one of ANSSI and BSI. First, they classify the capability to migrate to PQC by three roles (Urgent Adopter, Regular Adopter, Cryptographic Expert) representing the knowledge and capabilities of being able to use PQC. The proposed approach considers all cryptographic assets used in the organization and data, as well as the cryptographic assets of suppliers. Then the policy of identifying if the cryptographic asset is essential for the migration process is not only bound to the security level like for the approach of ANSSI and BSI, instead it depends on multiple factors:

- Key management of the asset
- Security goals connected to the asset
- Policies to use it in protocols and protocol versions
- Impact on the overall migration plan

2.2.2. Risk Assessment based Identification. An alternative to a policy-driven asset identification is a risk-based approach known from the domain of cybersecurity processes, like threat analysis and risk assessment (TARA) [18]. A risk-based identification approaches focus on establishing migration priorities based on the level of threat and potential impact on the organization. They involve linking the cryptographic asset inventory with a structured risk assessment in order to prioritize the items to be migrated. This approach is particularly relevant in a context where the resources available for migration are limited and where some assets have shorter vulnerability horizons than others.

One representative framework is the Crypto Agility Risk Assessment Framework (CARAF) [17]. This model proposes a five-phase approach:

- 1) *Definition of scope and threat vectors:* determine the scope of the analysis (applications, protocols, critical environments) and specify the type of threat considered, for example *store-now, decrypt-later* attacks made possible by the emergence of large-scale quantum computers.
- 2) *Structured inventory of cryptographic assets:* identify all elements (algorithms, libraries, certificates, keys, protocols, external services) involved in system security.
- 3) *Probability and impact assessment:* analyze the risk associated with each asset by combining its functional criticality (business impact, regulatory compliance) and its temporal exposure (lifespan of protected data, time required to perform a migration).
- 4) *Identifying mitigation strategies:* determine the possible actions for each asset (immediate mi-

gration, adoption of hybrid solutions, enhanced monitoring).

- 5) *Developing a road-map:* establish a prioritized plan aligned with the organization's business objectives.

Therefore, it follows a very similar approach like in the PQC Migration Handbook [4]. In the context of migration to PQC, [17] makes it possible to directly link cryptographic issues to enterprise risk management practices and to obtain prioritization based on both technical and organizational criteria.

At the same time, several strategic road-maps have been proposed to support the transition. The PQC Migration Roadmap [1], developed by its partners recommends, states in particular:

- 1) Conducting an inventory of the cryptographic mechanisms used in existing infrastructures,
- 2) Assessing their vulnerability to quantum attacks,
- 3) Implementing hybrid solutions during the transition phase,
- 4) Preparing for frequent updates as PQC standards evolve.

This document emphasizes the importance of prioritizing systems with long-term confidentiality requirements, i.e., those whose data must remain confidential for several decades.

These recommendations are supplemented by the road-map proposed by the European Commission, which highlights the need for collective and coordinated action [11]. This document emphasizes that migration cannot be achieved effectively by a single isolated entity, but requires collaboration between industry players, government agencies and standardization bodies. It highlights the importance of alignment between the availability of PQC standards, product life-cycles (particularly in embedded and automotive systems) and operational deployment constraints.

Finally, [10] provides a highly detailed operational framework in the form of an eleven-step iterative process. These steps include:

- 1) identifying architectures and compiling an inventory of assets (CBOMs),
- 2) constructing a dependency graph (Enterprise Dependency Digraph),
- 3) analyzing the vulnerabilities associated with each asset (Asset Vulnerability Reports),
- 4) assessing the criticality of dependencies,
- 5) prioritizing the elements to be migrated (Migration Priority Report),
- 6) and finally, developing iterative migration plans with step-by-step validation.

The key contribution of this document lies in its reproducibility: it provides a methodology that can be systematically applied to different organizational contexts, yielding comparable results.

2.2.3. Identification due to Vulnerability. The initial inventory-based identification of cryptographic assets that need to be migrated may not be enough. Certain components may seem to be of minimal significance from

a functional standpoint; however, they become a priority when they are associated with documented vulnerabilities, whether they are structural (insufficient key size) or implementation-related (flaws in libraries or protocols). The procedure outlined by ETSI in [10] accurately identifies vulnerabilities in cryptographic assets, including libraries and protocols, by utilizing a given dependency graph.

The ETSI model is based on the idea that the asset inventory (e.g., through a CBOM) must be enriched by mapping the vulnerabilities associated with each cryptographic element. This process, which is expressed in the Asset Vulnerability Reports, enables the reevaluation of the criticality of the identified components. ETSI's proposed process has a number of important steps. The initial step is to evaluate the inventory's correlation with existing vulnerabilities. Consequently, each cryptographic asset (algorithm, library, certificate, key, protocol) is correlated with vulnerabilities listed in a vulnerability database such as CVE². After this stage, the potential impact of each identified vulnerability on the classical security goals (availability, integrity, confidentiality) is analyzed, and the likelihoods are determined to obtain a standardized security metric for each identified asset. After the determination of the risk of the single cryptographic asset is identified, it is important to consider how this risk could propagate through the system. Therefore, the dependency of a cryptographic asset to other algorithms, protocols, and applications needs to be presented. ETSI models this by using a directed graph enriched with vulnerability annotations (*Enterprise Dependency Digraph*) to map dependencies. Depending on the overall threat to the system, a refined prioritisation is done in order to derive a mitigation priority plan for a structured transition procedure.

2.3. Approaches based on Dependency Graphs

Beyond approaches based solely on visibility (CBOM), a major area of research involves modeling cryptographic dependencies in the form of graphs. These approaches make it possible to represent complex relationships between components, detect critical structures (clusters, cycles) and guide migration strategies based on topological constraints. Three main contributions illustrate this approach: the formal model developed by Loebenberger et al. [16], the analysis framework proposed by Hasan et al. [14], and the operational methodology of ETSI, c.f. [10].

The formal model proposed by Loebenberger et al. [16] is based on a mathematical formalization using graph theory. In this model, each component to be migrated is represented by a node, while dependency relationships (communication, protocol, or regulatory) are modeled by directed edges. A directed edge from node v to node w means that a component v depends on w , which implies that v cannot be migrated without w being migrated first or simultaneously. This model notably makes it possible to identify "migration clusters," i.e., sets of closely related components that must be migrated together to avoid functional failure. A fundamental result of this approach is the demonstration that, in a system of size n , the number

of independent clusters is asymptotically of the order of $\frac{n}{\log(n)}$, highlighting that few components can be migrated in isolation. Other concepts are introduced, such as migration depth (successive chains of dependencies) and canonical strategies based on optimal topological scheduling. This model sheds light on the intrinsic complexity of migration, showing that any large-scale project will inevitably encounter numerous interdependent clusters.

Another dependency model is proposed by Hasan et al. [14] within a framework called *Security Dependency Analysis* (SDA). It directly links cryptographic relevant business assets to the cryptographic mechanisms that protect them. Their approach is based on three main steps: (i) creating an expanded inventory of cryptographic assets, including not only algorithms and libraries, but also databases, business processes, and critical services; (ii) constructing a graph where the nodes represent these assets and the edges represent security dependency relationships; (iii) analyzing criticality by combining the sensitivity of the business asset and the cryptographic robustness of the primitive used. Through this approach, certain components initially considered secondary become a priority because they protect data with high regulatory value or subject to long-term confidentiality constraints. Hasan et al. therefore enrich structural modeling with a business and regulatory dimension, filling a blind spot in purely technical models.

To the best of our knowledge is the third framework that focuses on dependencies within a guideline from ETSI. This ETSI TR104016 report [10] proposes an operational and iterative approach, centered on the construction of a directed graph for modeling a so-called *Enterprise Dependency Digraph*. This graph integrates both technical dependency relationships (applications, protocols, libraries, keys, certificates) and known vulnerabilities for each asset, documented in the Asset Vulnerability Reports. Circular dependencies (e.g., mutually signed certificates) are identified as *unbreakable clusters*, which can only be migrated as a whole. The ETSI approach comes with a set of standardized deliverables (enriched CBOM, dependency digraphs, vulnerability reports and prioritization reports) to ensure the reproducibility and auditability of the process. This pragmatic dimension makes it a tool that can be directly used by organizations wishing to plan and document their migration.

2.4. Preliminary Comparison of the Approaches

The approaches presented above offer complementary perspectives on cryptographic migration. Risk assessment models, such as in [17, 4], enable migration planning to be linked to organizational risk management practices. They provide a structured methodological framework but remain at a relatively abstract level. Roadmaps such as the *PQC Migration Roadmap* [4] or the coordinated roadmap proposed by the EU PQC Workstream [11] provide a strategic and collective vision, emphasizing the need for gradual preparation and interorganizational coordination. However, they do not go into detail about technical dependencies or practical implementation constraints. The ETSI TR104016 [10] report takes an operational perspective. It provides a reproducible and auditable methodology based on inventories (CBOM), dependency graphs and

2. <https://www.cve.org>

vulnerability reports. Its main interest lies in the production of standardized deliverables to guide and document migration at the organizational level. Nevertheless, this model remains dependent on the quality of the inventories and does not offer a mathematical formalism of complexity phenomena. The formal model developed by Loebenberger et al. [16] adopts a rigorous approach based on graph theory. It highlights the existence of migration clusters and demonstrates that any large-scale project has intrinsic complexity. This formalization is an important theoretical advance but requires exhaustive and precise knowledge of dependencies, an assumption that is often unrealistic in distributed and dynamic environments, where dependencies may be implicit. Finally, the framework proposed by Hasan et al. [14] introduces an additional dimension by directly linking business assets to the cryptographic primitives that protect them. Their Security Dependency Analysis approach allows migration to be guided by data sensitivity and regulatory requirements. It highlights the central role of business security, but relies on the availability of reliable inventories and the ability to effectively correlate technical and organizational layers. In summary, each of these approaches provides essential elements: CARAF and the road-maps offer a strategic, risk-oriented vision; ETSI provides a standardized operational methodology; Loebenberger offers a formal theoretical basis; and Hasan links migration to business and regulatory issues.

However, one blind spot remains: none of these approaches addresses in detail the importance of the management of cryptographic keys and its impact on the overall concept of system-level security. Yet key handling (generation, distribution, storage, revocation) is a critical dependency that is often implicit, particularly in embedded or distributed environments. In addition, all of these presented models assume that the identified assets and their dependencies are mapped into a correct and complete model, meaning that all relevant dependencies are automatically captured and represented by the inventory tools [26], [30], [27], [20] or via a CBOM [22] and the topology of the dependencies is correctly mapped or presented in a cryptographic dependencies sheet [8]. Hence, it is not assured that all dependencies are directly captured just by analyzing the software or architecture topology or the communication traffic regarding the usage of cryptographic primitives. Dependencies that arise from security concepts or key dependencies are therefore not covered by the above-mentioned approaches.

The lack of explicit consideration of this dimension weakens the robustness of existing models and justifies the development of approaches that explicitly integrate key management and its systemic effects. Therefore, we will present and discuss a formal framework for modeling various types of dependencies (obvious and non-obvious dependencies). Furthermore, we will introduce in the following section, based on the formal framework, methods to identify additional dependencies and thereby extend dependency graphs by means of the dependency between the identified assets.

3. Description of the Use Case Example

For studying the application of our proposed concept, we investigated the transformation of a small exemplary invehicle architecture depicted in the lower right corner of Fig. 3 and Fig. 2. This experienced-based example illustrates an automotive scenario with security controls that are commonly used in practice still today. In both cases, the example system is based on two simplified components and one external simplified component (a server backend). The two invehicle components are a telecommunication unit (TCU) and a general basic ECU. The TCU is the only component in this architecture that can directly communicate with the backend server via a remote communication channel via TLS 1.2. The communication between the ECU and the TCU is done via an internal control area network (CAN) bus. Therefore, the ECU can exchange data with the server via the TCU. Thereby, this assembles a very simplistic example of a vehicle architecture of the typical domain-specific architecture, which is still the most common architecture for small and medium-size combustion engineering cars. It is a simplified example of the domain-oriented architecture, because usually the amount of ECUs is not two like in this example; usually it is between 50 and 80 ECUs in a small- to medium-sized car.

The primary focus of the subsequent discussion and description will be the automotive components ECU and TCU for dependency monitoring.

3.1. ECU Component

Every ECU inside a car has some specific vehicle-relevant functions contributing to the overall functionality of a car; for instance, an ECU for controlling the brakes so that the car can decelerate or be put to a stop in a controlled manner via the braking pedal on the driver's side. ECUs also have basic maintenance and service functions implemented alongside security controls to ensure the overall functionality of the ECU in addition to their specific functions.

The functionality of the ECU in this use case study is very limited and concentrates only on three application functions:

- Software update manager, which handle the software update of the ECU
- Diagnosis services, which provides a maintenance interface to the ECU for checking and maintenance tasks of the ECU
- Invehicle Communication, which is a ECU internal service of the secure can communication stack

This application, are secured by ECU specific security controls or also use services of security controls to ensure a safe and secure execution of the functionality of the application.

In our example of our use case, the integrity and authenticity of all applications are secured by the security control secure boot. This security control verifies the complete integrity and authenticity of the software of the ECU using an RSA signature scheme with a key length of 2048 bit in the first boot stage and an HMAC-256 in the second boot stage. In practice, a symmetric scheme

is used as an integrity check for the software due to verification speed reasons. Hence, the first secure boot stage uses an asymmetric scheme and verifies only a small amount of data, usually the cryptographic primitives and the related keys. Since classical ECUs³ have a monolithic, statically built firmware, the secure boot not only verifies the correctness of the application and the other security controls, but also the integrity and authenticity of other keys stored in the firmware and used for other security controls. Hence, if an attacker were to replace the private and public key pair, in this example K_{pr2} and K_{pb2} , he or she could manipulate the software as well as replace all the other keys of the security controls. In this case, the integrity of the other security controls depends on the integrity of the key pair of the secure boot security control, cf. Fig. 3. The ECU's internal dependency is not immediately apparent without domain knowledge. Moreover, the software topology alone doesn't provide enough information to extract this dependency, which is why most inventory tools cannot capture it when generating CBOMs, cf. Fig. 2.

The security control secure onboard communication is responsible for ensuring authentic communication via the CAN bus. Therefore, each CAN frame is secured via a truncated MAC that is used on the receiving side for the verification of the authenticity and integrity of the CAN message, compliant with the AUTOSAR standard Secure OnBoard Communication [6]. Following the standard, the MAC is generated using AES-128 CMAC with a key length of 128 bits, in our example using K_1 . Since the ECU should be able to communicate authentically via the CAN bus with the TUC employing the AUTOSAR specific Secure OnBoard Communication protocol, the TCU needs to have knowledge of the value of the secret key K_1 . The corresponding key stored inside the TCU used for the secure Onboard communication is K_8 ⁴. Therefore, there is a cross device not obvious dependency between K_1 inside the ECU and K_8 inside the TCU, cf. Fig. 3 and Fig. 2, respectively.

Another security control of the ECU is secure software update which should ensure in combination with the security control secure access the only authentic software can be installed by authorized personnel. In this example, the authenticity and integrity of the software of the update packages is verified by an AES-128 CMAC using K_5 ⁵. Since, in our example, the software update package is generated and sealed by the server backend, there is an obvious dependency between K_5 inside the ECU and K_{17} inside the simplified server backend, cf. Fig. 3.

However, before the update procedure, including the security control secure software update, is executed, the access to the memory must be granted via the security control secure access. This security control uses a symmetric challenge response scheme based on AES-128 with key

K_4 as authorization control so that only a user with the knowledge of K_4 gets access to the memory of the ECU and is allowed to read and write the memory via services. Therefore, is an attack would get know or get control over K_4 he or she can change every bit of the firmware on the ECU via maintenance service and direct write and read access to the memory. Hence, the confidentiality and integrity of all the other keys stored inside the ECU depend on the confidentiality of K_4 , cf. Fig. 3.

3.2. TCU Component

For our use case study, we strongly focus on the ECU; therefore, we only focus on the security controls of the TCU. The TCU has the same security controls as the ECU, using the same cryptographic primitives, but also extends the security controls by secure remote communication. This security control is used to establish a secure communication channel with the server backend via TLS1.2, cf., Fig. 2. Consequently, similar to the ECU case, no obvious dependencies between the key for the first stage secure K_{pr7} and the other keys within the TCU, nor between the security access control (security access) K_9 and the other keys within the TCU.

Due to the security control, secure remote communication, and the TLS1.2 primitive, additional cryptographic primitives exist in the TCU that do not exist inside the ECU. Hence, additional keys exist with also not obvious dependencies, especially with a strong cross-component dependency towards the server backend. Therefore, there is a non-obvious dependency between K_{pb12} inside the TCU and K_{pb16} for the signature verification step during the TLS handshake. A second non-obvious dependency between keys exists regarding the Diffie-Hellman key agreement step for the TLS handshake, e.g., K_{pb10} inside the TCU and K_{pr14} from the server side.

The secure software update security control of the TCU is different compared to that of the ECU. In the case of the TCU, the authenticity of the software is verified by using an RSA signature scheme using a 2048-bit-long key. Therefore, there is another cross-component dependency regarding the public keys K_{pb13} on the TCU side and the private key K_{pr18} on the server side, cf. Fig. 3.

4. Basic Definitions of the Dependency Model

Following [16], we model a cryptographic migration project as a directed graph $G = (V, E)$ where V is the set of nodes, and $E \subseteq V \times V$ is the set of edges. This graph is called a *migration graph*, and it captures the components to be migrated and their (direct) dependencies. Each node represents a migration component, i.e., an element of the infrastructure to be migrated. Components can be of different types: cryptographic software libraries, complete software packages, operating systems, network protocols (TLS, SSH, IPsec, etc.), cryptographic hardware, external services or APIs used by the system, or even standards systems comply to. The granularity (the level of detail at which one chooses to model an infrastructure element as a single node in the graph) of the nodes depends on the context and the modeling objective. The granularity of the nodes considered in [16] is coarse – a node represents

3. ECUs using AUTOSAR Classic [29] as basic software platform

4. In our example, the corresponding key K_8 for secure OnBoard Communication from TCU fitting to the key on the ECU K_1 has a different index, because usually the devices are not necessarily manufactured by the same company and thus are interpreted as individual instances.

5. Using symmetric schemes to verify the authenticity via a sealing approach is customary in the automotive domain due to the resource constraints and the verification speed.

an entire high-level component, e.g., operating systems, network protocols (TLS, SSH, IPsec).

The edges (v, w) in the set E model the *migration dependencies*. We also write $v \rightarrow w$ instead of the pair (v, w) . An edge $v \rightarrow w$ is interpreted as “component v cannot be migrated before w , but could potentially be migrated at the same time”. These dependencies can be technical (an application that depends on a library), protocol-related (version compatibility between communicating systems), regulatory (updates conditioned by a standard or law), or related to performance and hardware resources.

A central concept is that of a migration cluster $c(v)$, defined as the set of components that are mutually dependent on v . In graph theory, this is exactly a strongly connected component⁶. All elements of a cluster must be migrated simultaneously, otherwise service disruptions may occur. A cluster can be reduced to a single autonomous node or contain several highly interconnected components. A component (or cluster) is said to be *migratable* if it has no outgoing dependencies.

To plan the migration, one can condense the graph into a cluster graph G/c . In this graph, each node represents a cluster, and an edge $c(v_1) \rightarrow c(v_2)$ exists if an element of $c(v_1)$ depends on an element of $c(v_2)$. By construction, G/c is acyclic (a graph without cycles), which allows for the calculation of a topological order. This order determines the migration strategies, migration length $\ell(G)$ (number of clusters to migrate) and the migration depth $d(G)$ (maximum length of a chain of dependencies between clusters). A canonical migration strategy consists of migrating all clusters of the same topological level in parallel, progressing from level 1 to the last. See Section 3.4 in [16] for details on this.

The migration process in the model follows four steps: (1) identify a migratable component or cluster according to the chosen strategy, (2) determine its complete cluster, (3) migrate this cluster and remove it from the graph along with its edges, and (4) repeat until the graph is empty. A fundamental lemma guarantees that there always exists at least one migratable component in any non-empty finite graph (cf. Lemma 1 in [16]).

The authors of [16] also identify recurring patterns in these graphs: isolated components without dependencies, multiple dependencies on a single element, dependencies of one element on several others, dependency cycles (often with two nodes, rarely larger), regulatory or performance constraints, and classic acyclic graphs for software dependencies.

While this model provides a solid formal basis for representing and analyzing cryptographic migration projects, it has a central limitation when it comes to capturing the real complexity of infrastructures: The model assumes that all existing, migration-relevant dependencies between components are known. However, in practice, this is often not the case!

We thus distinguish here between the known, explicit dependencies and the unknown, implicit dependencies. In order to be able to detect implicit dependencies, we need a

more fine-granular view on the involved components. We thus enrich the graph formalization by considering that cryptographic keys themselves, algorithms, certificates, and all other cryptographic primitives can be modelled as nodes. This changes the granularity of the representation. It becomes fine-grained: a node v in the authors’ model, representing a single component, can now correspond in our model granularity to a collection of several finer-grained nodes (e.g., the associated key, the algorithm, and the services that use it).

In the original model, a migratable cluster is defined solely by explicit dependencies: it is migratable if no outgoing edge connects it to another non-migrated cluster. But in our extension, a cluster can appear migratable from the explicit perspective while actually being blocked by an implicit dependency, calling into question the security or feasibility of its isolated migration.

This structural difference therefore requires a redefinition of cluster formation and validation rules, as well as an adaptation of migration strategies to account for these hidden links. It is precisely this extension that we introduce into our formalization, in order to better represent the operational reality of complex cryptographic migrations.

5. Extensions of Basic Dependencies Models

The graph-based formalization explained in the previous section considers a directed graph $G = (V, E)$ explicitly representing the logical structure of a cryptographic system in an architecture, which can be derived from a CBOM representation [22] or from inventory tools like [26]. We assume that the set of migration components V is fixed for this model, whereas the set of edges E can evolve. The initial set E contains all explicit dependencies. Further inspection reveals implicit dependencies and enriches E in an iterative-incremental process. The following subsection describes this process. The subsequent subsection gives an overview on dependency types and provides thereby a basis for goal-oriented inspection and detection of implicit dependencies.

5.1. Modeling Explicit Model Refinement

We are now going to model explicit model refinement over successive steps. In a given model at a given step, k the graph $G_k = (V, E_k)$ contains the *explicit* dependencies given by E_k . We assume here that the set of vertices of the graphs do not change. We assume further that there is a set $\bar{E}$ of all dependencies which are relevant for migration, or *relevant dependencies* for short. As described in the previous section, the initial set $E_0 := E$ can be e.g., derived from a CBOM, and the ultimate goal is to reach $\bar{E}$ by iterative-incrementally detecting and adding implicit edges.

The edges $I_k := \bar{E} \setminus E_k$ that have not yet been discovered are referred to as implicit edges. Note that by construction $I_k \cap E_k = \emptyset$ dependencies are either explicit or implicit, but never both. We will elaborate in Section ?? on how to algorithmically identify implicit dependencies.

Specifically, a valid *refinement* of the graph G_k is a graph $G_{k+1} = (V, E_{k+1})$ with implicit dependencies I_{k+1} if $I_{k+1} \subseteq I_k$ and $E_{k+1} = E_k \cup (I_k \setminus I_{k+1})$, i.e., a graph in

6. a strongly connected component (SCC) of a directed graph is a maximal sub-graph in which each pair of vertices is connected by a path in both directions. In other words, for any pair of vertices u and v in an SCC, there exists a path from u to v and a path from v to u .

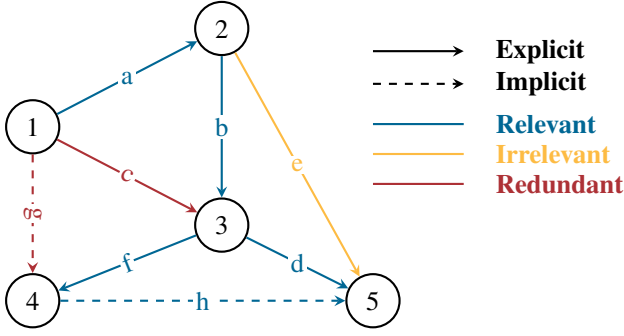


Figure 1. Dependency graph illustrating explicit (solid) and implicit (dashed) dependencies among five artifacts. In the picture, the explicit dependencies a , b , d and f imply redundant dependencies c and g . Dependencies e are irrelevant if its removal does not imply an implicit dependency and h is implicit-relevant. In the refinement process h needs to be made explicit. By definition, implicit-irrelevant dependencies do not occur.

which some of the previously implicit dependencies got explicit.

There may exist *irrelevant dependencies*, as the sets $E_k \setminus \bar{E}$ are potentially non-empty. An irrelevant dependency can also be defined as an explicit dependency $u \rightarrow v$ whose removal does not add an implicit dependency $u \rightarrow v$ to the graph. Irrelevant dependencies “pollute” in the migration graph and can lead to suboptimal migration strategies.

There is a third type of dependencies, *redundant dependencies*. A dependency $u \rightarrow v$ is redundant if it can be substituted by a path $u \rightarrow^* v$ ⁷ of explicit dependencies connecting the same nodes. We illustrate the different dependency-types in Figure 1.

Further examples for the different dependency types can also be found in the use case depicted in Fig. 2 and Fig. 3: The solid arrows depict explicit dependencies, e.g., in the ECU the connection between the SW Update Manager and the Secure SW Update, whereas dashed arrows depict implicit dependencies, e.g., the bidirectional shared key connection between K_1 in the ECU and K_8 in the TCU. The connection between Secure SW Update and Cryptolib in the ECU is redundant, as there exists a path from Secure SW Update via Secure Access to Cryptolib. The use case does not contain any irrelevant explicit dependencies.

If G_{k+1} is a refinement of the graph G_k in the above sense, we write $G_k \preceq G_{k+1}$. The whole refinement process

$$(V, E) = G = G_0 \preceq G_2 \preceq \dots \preceq G_m = (V, \bar{E})$$

starts with a dependency graph G_0 and refines the graph until G_m it contains all relevant dependencies.

5.2. Taxonomy of dependencies

Beyond the previous classification (explicit/implicit, relevant/irrelevant, redundant), dependencies can also be characterized according to their functional nature within the system. Generally, whether explicit or implicit, dependencies are based on two fundamental relationships:

7. The notation $u \rightarrow^* v$ means that there is a transitive path from u to v via the relation $\rightarrow$.

dependsOn and *provides*, which respectively describe the links of use and provision of services. Explicit dependencies constitute only the visible instances of these relationships, as they appear in architectural models or documentation. These relationships allow dependencies to be grouped into several functional subcategories [16, 22].

Protocol dependencies. They link a protocol to the algorithms and cryptographic hardware it uses. For example, in Fig. 2, TLS 1.3 relies on ECDH for key exchange, AES-GCM for encryption, and SHA-256 for integrity. In a distributed context, a high-performance computing (TCU) environment may implement TLS 1.3 with a specific configuration, while a remote server uses the same version of the protocol with potentially different cryptographic suites. Analyzing these dependencies makes it possible to identify commonalities or differences between configurations and assess the systemic impact of a vulnerability: compromising a primitive weakens the entire protocol and, consequently, all entities that use it.

Functional dependencies. Functional dependencies correspond to direct technical relationships in which one component relies on another to perform its cryptographic operations. They are generally the most visible and easiest to identify, as they often appear in SBOM or CBOM inventories. A classic example is an application that depends on a cryptographic library, such as OpenSSL, to perform encryption, signing, or hashing operations. In such a case, the application can only be migrated to post-quantum algorithms if the underlying library already supports these new algorithms. Structurally, these relationships are often represented in a Cryptographic Bill of Materials (CBOM) by *dependsOn* pairs, which explicitly specify that a component A can only function in the presence of a component B .

Figure 2 (from our study; detailed in Sect. 3) highlights these functional dependencies, shown in yellow. It shows how application layers, cryptographic libraries and algorithms interlock. This visual representation illustrates:

- the potential depth of dependency chains,
- the need to analyze each relationship to avoid incompatibilities,
- the fact that migration cannot be reduced to a simple substitution of algorithms, but must be designed as a coordinated sequence of transformations.

Communication dependencies. Communication dependencies concern relationships between components that exchange data protected by standardized cryptographic protocols, such as TLS. Unlike functional dependencies, which describe an internal link for using a library, communication dependencies are expressed through an *interoperability* constraint between communicating entities. In CBOMs [22], communication dependencies can be made visible using the *provides* fields, which describe the services or algorithms offered by a component. For example, a cryptographic library may provide a set of primitives, while other components express their dependency via *dependsOn* relationships. This model allows reasoning not only in terms of files or packages, but more importantly in terms of the cryptographic services actually exchanged.

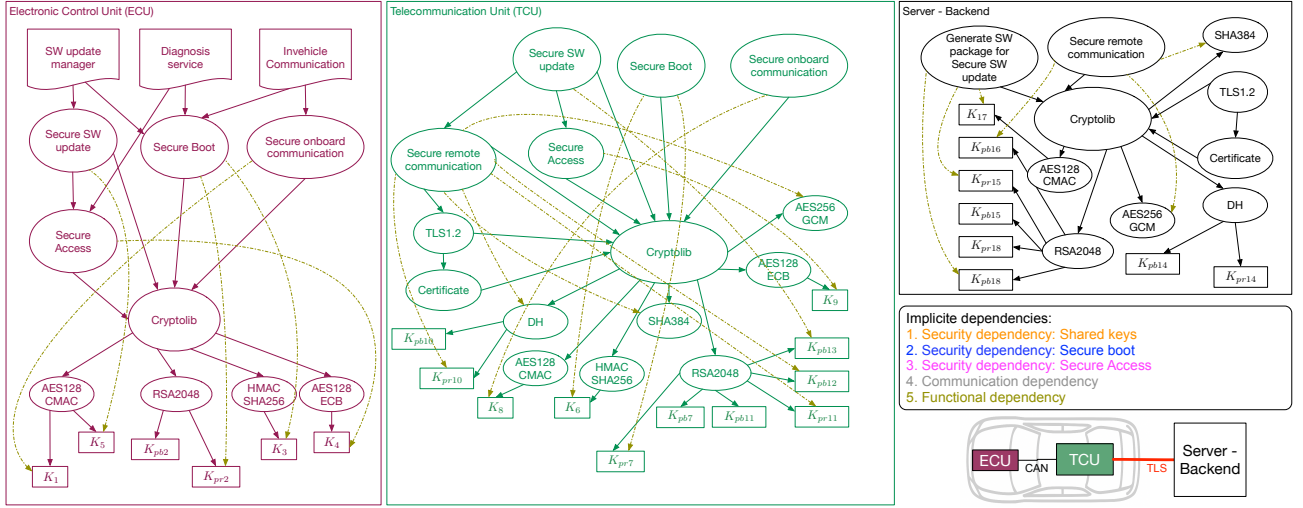


Figure 2. Graph model of the use case example with the topological dependencies of all system and software components (derivable from a CBOM - solid arrow lines) and functional dependencies (yellow dash arrow lines) and a schemed architecture overview in the lower right corner of the figure. The information represented in this dependency graph can be derived from a CBOM following the CycloneDx format [22].

Among these visible dependencies, two subcategories are particularly relevant: functional dependencies and communication dependencies. The integration of these visible dependencies into a graph model such as the one proposed in [16] makes it possible to simulate migration scenarios, calculate optimal sequences, and assess the risks associated with different transition paths. However, this modelling, however rigorous it may be, does not cover all operational realities. Many critical relationships escape explicit representation. This is where the concept of implicit dependencies comes into play.

implicit dependencies refer to relationships between cryptographic components that are not formally expressed in dependency graphs or CBOMs. They result either from poorly documented internal mechanisms, dynamic configurations, or complex environmental relationships. Their invisibility makes them all the more dangerous: they cannot be addressed if they are not identified. Yet they are often the cause of migration failures, loss of functionality after deployment, or breaks in the chain of trust.

We have proposed a three level classification of these dependencies, based on their scope and nature: intra component, inter component, and external.

Intra-component dependencies. Those dependencies concern internal relationships within a component of the architecture inside the vehicle. In the architecture under consideration in Fig. 3, they manifest themselves in particular in the HPC through the secure boot mechanism. This depends on the private key K_{pr7} , used to verify the integrity of software images. On the graph, the blue edges leading to K_{pr7} represent these functional dependencies: all other local keys are indirectly involved in the secure boot process. A similar situation can be observed in the ECU, where the private key K_{pr2} plays a central role in secure boot. In addition, the pink edges indicate internal dependencies related to the secure access function, which relies on the K_4 key and its interaction with other local keys. These dependencies, although crucial, are often implicit because they are deeply embedded in hardware components and do not appear explicitly in CBOMs. They are so fragile that a unilateral modification of the key or

the associated algorithm can render the device unusable.

Inter-component dependencies. When at least two main components of the architecture share the same trust element, an inter-component dependency arises from that. In the case of the vehicle, these implicit links unite, for example, the HPC and the ECU around an identical shared key (K_8 and K_1) Fig. 3, represented in the graph as a common resource. Updating one of these keys or their algorithms in one component requires immediate and consistent propagation to the other component, otherwise the chain of trust is broken and internal communications fail. These dependencies are difficult to detect because internal chains of trust are not always documented in CBOMs and may depend on proprietary implementations. However, they pose a major risk to the consistency and continuity of security in the vehicle.

External dependencies. Those dependencies concern the links between an internal vehicle component (HPC or ECU) and external infrastructure, typically a remote server. In the graph example in Fig. 3, secure communication between the HPC and the server is based on TLS 1.2, with specific certificates and keys (e.g. K_{pr14} on the server side). These dependencies are particularly critical because they are beyond the direct control of the internal organization: the server can unilaterally decide to modify its certificates, migrate to a post-quantum algorithm or change its cryptographic policy. If the embedded components (HPC or ECU) do not keep pace with these developments, interoperability disappears and services fail. These external dependencies therefore require continuous monitoring mechanisms, cryptographic agility and, in many cases, contractual coordination with the third parties involved.

These three levels of implicit dependencies cannot be modeled solely using standard tools such as CBOMs. Identifying them requires in depth documentation and testing in a real environment. Including them in a migration model requires enriching traditional graphs with specific annotations, for example in the form of co migration constraints, implicit prerequisites, or contextual dependencies. This illustration supports the hypothesis

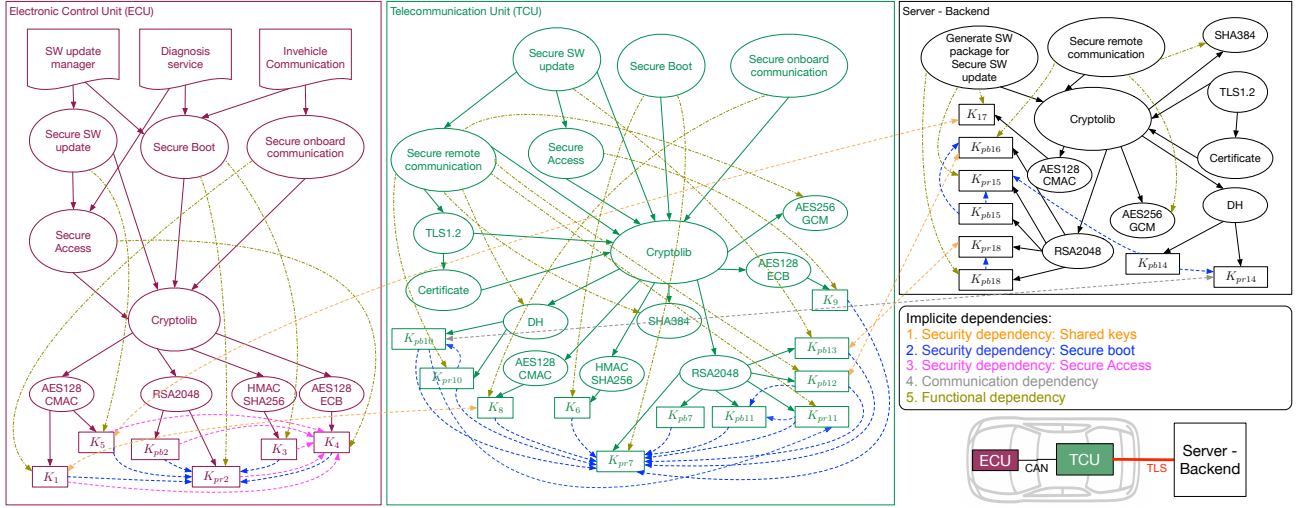


Figure 3. Graph model of the use case example enriched with different type of dependencies: topological dependencies of all system and software components (derivable from a CBOM - solid arrow lines) and non-obvious or implicit dependencies (dash arrow lines).

that a migration model must combine explicit analysis tools (graphs, CBOM) with a more in depth analysis, integrating implicit dependencies linked to internal mechanisms, cryptographic life cycles, and interorganizational relationships.

6. Formalization and Generation of Migration Strategies.

In order to move beyond the conceptual analysis of dependencies required for migration, this section gives a general formal framework for the formulation, analysis, and verification of migration strategies. The validity of the framework is illustrated by the application of it to the analysis of an exemplary algorithm.

We denote the system under consideration, i.e., a system subject to migration as $\mathcal{C}$. $\mathcal{C}$ is characterized by a migration graph $G = (V, E)$, a migration function $\mu : V \rightarrow V'$, an execution oracle ζ and the test functions P . G and $\bar{E}$ have already been formalized and described in Sec. 4 and Sec. 5. μ , ζ and P are formalized in this section. On the basis of these concepts, we then formulate and analyze a clustering and detection algorithm that yields a valid migration strategy.

6.1. Migration

The bijective function $\mu : V \rightarrow V'$ models the concept of uniquely assigning a PQC counterpart (migrated node) $v' := \mu(v)$ to each $v \in V$ (unmigrated node).

For a set, $W \subseteq V$ the term $\mathcal{C}[W]$ denotes the system $\mathcal{C}$ with all nodes $v \in W$ being replaced by their PQC counterpart $v' = \mu(v)$. For a sequence of sets $[W_1, \dots, W_\ell]$ with $W_i \subseteq V$, the term $\mathcal{C}[W_1] \dots [W_\ell]$ denotes the system $\mathcal{C}[\cup_{i=1}^\ell W_i]$.

6.2. Oracle

ζ models the concept of running a system and checking for errors. This will be the basis (see Sec. 6.4) for

defining the set of all relevant dependencies $\bar{E}$, i.e., all dependencies which are relevant for migration.

The application of ζ to a system S is denoted by $\zeta(S)$. S can be the original system $\mathcal{C}$ or a (potentially partially) migrated system $\mathcal{C}[W]$. Two cases are possible:

$$\zeta(S) = \begin{cases} 0 & \text{(system failure)} \\ 1 & \text{(system is functional)} \end{cases}$$

We assume that the oracle behaves as follows:

- **Hypothesis 1: No resource constraints.** The oracle does not model any hardware limits: if a migrated system fails, it is never due to a lack of memory, computing power, or execution capacity.
- **Hypothesis 2: System determinism.** For a given system state, the oracle will always return the same answer. If a migrated system is functional, resp. fails, we will get the same answer for the same system when asking the oracle at another time.
- **Hypothesis 3: Any system failure that occurs after migration reveals an unidentified dependency.** A failure always means that a relevant dependency has not being met (a migration must be performed first). In the case of mutual relevant dependencies, this means that two nodes have to be migrated together.

We therefore assume, that an oracle ζ satisfies:

- The oracle ζ behaves like a function.
- $\zeta(\mathcal{C}) = 1$ (valid empty migration)
- $\zeta(\mathcal{C}[V]) = 1$ (valid total migration)

6.3. Test functions

The purpose of the test function is to detect all relevant dependencies between the system components. It constitutes an analysis layer that identifies implicit, hidden, or invisible relationships that do not appear in the explicit migration graph $G = (V, E)$. Each system inspection activity is represented by a partial test function. A partial test function evaluates whether a particular type of dependency exists between two nodes. Test can detect functional

links, key sharing, dependencies related to secure boot, or interactions with an external server, etc. (cf. also Sec. 5.2).

Let V be the set of system components. We define a set of partial test functions: $\mathcal{T} = \{f_1, f_2, \dots, f_k\}$ where each $f_t : V \times V \rightarrow \{0, 1\}$ corresponds to a specific dependency category (cf. also Sec. 5.2). So the test function is the global test predicate P defined by:

$$P(v, w) = \begin{cases} 1, & \text{if } \bigvee_{f \in \mathcal{T}} f(v, w) = 1, \\ 0, & \text{otherwise.} \end{cases}$$

The set of relevant dependencies detected is then:

$$\bar{E} = \{(v, w) \in V \times V \mid P(v, w) = 1\}.$$

Definition 6.1 (Consistent test function). A test function P is consistent if and only if it is reflexive and transitive. In fact, reflexivity expresses that a component necessarily possesses all the properties evaluated by a test: whether it is a functional, cryptographic, or structural property, any inspection applied to a node as itself must naturally be true. Transitivity, on the other hand, models the propagation of implicit dependencies: if a test detects that u depends on v (for a key, a service, or a resource), and that v itself depends on w for the same reason, then u necessarily depends on w . Thus, transitive closure ensures that the detected dependencies accurately reflect the system's actual functional structure.

Example. Let: $f : V \rightarrow K$ where $K \subset V$ denotes the set of nodes representing keys. The function f associates each node v with the key k it uses. We then define the key sharing test function as follows:

$$P(v, w) = \begin{cases} 1 & \text{if } f(v) = f(w) \\ 0 & \text{otherwise} \end{cases}$$

In other words, the value $P(v, w) = 1$ means that the two nodes v and w use exactly the same key.

In the specific case of the key sharing test defined above, consistency is immediate:

- reflexivity $P(v, w) = 1$ is verified since each node uses the same key as itself, i.e., $f(v) = f(w)$;
- transitivity $P(u, v) = 1$ and $P(v, w) = 1$ implies $f(u) = f(v) = f(w)$, therefore $P(u, w) = 1$.

Thus, the key sharing test is indeed a consistent test function

6.4. Migration Strategy

The ultimate goal of a migration algorithm is to identify a valid migration strategy. A migration strategy $\mathcal{M}$ is a sequence $(C_1, \dots, C_\ell)$ of disjoint sets $C_i \subseteq V$ for $i = 1, \dots, \ell$. A migration strategy is valid if the migration sequence $C_1, \dots, C_\ell$ does not fail. The concept of a valid migration strategy is formalized as follows:

Definition 6.2 (Complete migration strategy). A migration strategy $\mathcal{M} = (C_1, \dots, C_\ell)$ is complete, if $\bigcup_{i=1}^{\ell} C_i = V$.

Definition 6.3 (Valid migration strategy based on oracle). Let $\mathcal{M} = (C_1, \dots, C_\ell)$ be a migration strategy. We recursively define a sequence of systems:

$$S_0 := \mathcal{C} \quad (\text{no migrated nodes}),$$

$$S_k := S_{k-1}[C_k] \quad (\text{migrate nodes } C_{k+1})$$

$$\text{for } k = 1, \dots, \ell,$$

The strategy $\mathcal{M}$ is said to be valid if and only if:

$$\zeta(S_k) = 1 \quad \forall k = 1, \dots, \ell$$

In other words, at each step, the system obtained after the successive migration of $C_1, \dots, C_k$ must be declared functional by the oracle.

Example. Based on the architecture of Fig. 3, let C_1 , C_2 , and C_3 represent respectively the ECU, the HPC, and the Server clusters. The migration strategy $\mathcal{M} = (C_1, C_2)$ is not valid, since both ECU and HPC share an implicit dependency through the key K_8 used for secure in-vehicle communication. Migrating C_1 before C_2 breaks this hidden link and renders the system non-functional. Consequently, e.g., strategies containing (C_1, C_2) as a subsequence (e.g., (C_1, C_2, C_3) or (C_3, C_1, C_2)) are also invalid.

Lemma 6.1 (Bad Prefix). Let $\mathcal{M} = (C_1, \dots, C_\ell)$ be an invalid migration strategy. Then any migration strategy $\mathcal{M}' = (C_1, \dots, C_\ell, C_{\ell+1}, \dots, C_{\ell'})$ is also invalid.

Proof. The proof follows immediately from definition 6.3. $\square$

6.5. Dependency Types

The edge set E in the migration graph $G = (V, E)$ contains all initially known dependencies, which we have denoted as explicit dependencies in Sec. 5. Based on the concept of valid migration strategies and in accordance with Sec. 5 we can now define the concepts *relevant* dependency and *implicit* dependency.

Definition 6.4 (Relevant dependencies). An edge $u \rightarrow v \in V \times V$ is a relevant dependency if there is no valid migration strategy $\mathcal{M} = (C_1, \dots, C_\ell)$ with $u \in C_i$ and $v \in C_j$ and $i < j$. The set $\bar{E}$ is defined as the set of all relevant dependencies.

In other words, the set $\bar{E}$ contains all edges $u \rightarrow v$ where it u cannot be migrated before v .

Definition 6.5 (Implicit dependencies). Any edge $u \rightarrow v \in \bar{E} \setminus E$ is called implicit dependency.

Example. Explicit versus implicit dependencies:

- **Explicit dependency** In Fig. 3, the *Secure boot* service explicitly depends on the cryptographic library *Cryptolib*, which implements HMAC SHA256. This relationship is represented in the CBOM by a visible edge $(\text{Secure boot}, \text{Cryptolib}) \in E$, and satisfies the condition $P(\text{Secure boot}, \text{Cryptolib}) = 1$, because the migration of *Cryptolib* conditions that of the service. It is therefore an explicit dependency.
- **Implicit dependency** In Fig. 3, the ECU's *Secure Boot* mechanism relies on a closed chain of trust between the private key K_{pr2} , and different keys: the public key K_{pb2} , K_1, K_2, K_3, K_4, K_5 . These internal links do not appear in the graph E , but the test function $P(K_{pb2}, K_{pr2}) = P(K_{pb2}, K_1) \dots =$

1, and the oracle ζ indicate that an isolated migration of one of these keys before K_{pr2} invalidates the system. This relationship, which is not represented in E , thus constitutes an implicit dependency.

6.6. Consistency

$\bar{E}$ denotes the theoretical concept of all dependencies which are relevant for migration. The oracle and the test function model two complementary practical approaches for detecting all dependencies $u \rightarrow v \in \bar{E}$. The test function models (static) analysis activities, whereas the oracle models the run-time-validation of the system (dynamic analysis). The oracle and the test function should be consistent with each other, i.e., agree on the set $\bar{E}$.

Let us consider a migration strategy $\mathcal{M} = (C_1, \dots, C_\ell)$.

Definition 6.6 (Consistency of ζ and P). The test predicate P and the oracle ζ are consistent with each other if and only if $\forall v, w \in V : P(v, w) = 1 \iff$ there is no valid migration strategy $\mathcal{M} = (C_1, \dots, C_\ell)$ with $v \in C_i$ and $w \in C_j$ and $i < j$.

Remark (Sufficient Consistency Condition). The validity of the condition

$$\forall v, w \in V : P(v, w) = 1$$

$$\iff$$

there is no valid migration strategy $\mathcal{M} = (C_1, \dots, C_\ell)$

with $v \in C_i$ and $w \in C_j$ and $i < j$

is sufficient for the validity of the algorithm in Sec. 6.7.

6.7. Clustering and Detection algorithm

Definition 6.7 (Transitive reduction). Given a directed graph $G = (V, E)$, the transitive reduction of G is the smallest graph $G_{\text{red}} = (V, E_{\text{red}})$ such that G and G_{red} have exactly the same reachability relation. An edge (u, w) is removed whenever there exists a path $u \rightsquigarrow v \rightsquigarrow w$ that makes the direct edge redundant. Thus, the reduction eliminates unnecessary edges while preserving the dependency structure of the graph.

Algorithm 1 describes the procedure used to traverse the migration graph G and to detect implicit dependencies. It relies on

- 1) a depth-first search (DFS) to examine each pair of nodes and dynamically enrich the set of dependencies
- 2) and subsequent clustering of co-dependent nodes
- 3) the systematic use of transitive reduction to maintain a migration graph that is both correct and redundancy-free.

The clustering is implemented via graph condensation, as described in Sec. 4, and in our algorithm, the transitive reduction plays a crucial role at two points:

- immediately after the construction of the initial migration graph E ,
- and on the condensed graph E/c after each discovery of an implicit dependency.

This guarantees that the SCCs and the choice of the next migratable cluster are based only on essential direct dependencies, without pollution from redundant edges.

In order to show the correctness of the algorithm, we have to prove:

- that the algorithm always terminates after a finite number of steps;
- that the generated sequence is correct, meaning that each migration step validated by the algorithm is indeed acceptable for the oracle ζ ;
- that the sequence is complete, that is, that it covers all nodes: $\bigcup_{C \in \mathcal{S}} C = V$.

Lemma 6.2 (Termination). Under the assumption of joint consistency between P and ζ The migration algorithm terminates after a finite number of iterations.

Proof. see A.1 □

Lemma 6.3 (correction). Under the assumption of joint consistency between P and ζ . At any point during the execution of the algorithm, the migration sequence $\mathcal{S}$ constructed so far is valid, that is:

$$\zeta(\mathcal{C}[\mathcal{S}]) = 1.$$

Proof. see A.2 □

Lemma 6.4 (Completeness). Under the assumption of joint consistency between P and ζ The final migration sequence $\mathcal{S}$ covers all nodes:

$$\bigcup_{C \in \mathcal{S}} C = V.$$

Proof. see A.3 □

Theorem 6.5 (Complete correctness of the algorithm). Under the assumption of joint consistency between P and ζ , the algorithm 1:

- terminates after a finite number of iterations;
- returns a sequence $\mathcal{S}$ such that each prefix is validated by ζ (correctness);
- the sequence $\mathcal{S}$ covers all nodes V (completeness).

Consequently, the algorithm terminates with a migration strategy *valid and complete*.

Proof. see A.4 □

7. Conclusion

This paper argues that large scale PQC migration requires going beyond coarse component inventories and explicitly modeled protocol dependencies. We therefore introduce a dependency centered model that refines initial migration graphs by treating cryptographic primitives, keys and security controls as first-class nodes, thereby also making otherwise implicit dependencies visible.

Building on the graph model of [16], we enrich the notion of migration graphs with a taxonomy for migration related dependencies (explicit, implicit, relevant, redundant dependencies) and a taxonomy based on functional nature (protocol, functional and communication dependencies as well as intra-component, inter-component and external implicit dependencies). The second taxonomy guides the

Algorithm 1: Cluster migration with implicit dependency detection

Input:

- 1) directed graph $G = (V, E)$
- 2) test function $P : V \times V \rightarrow \{0, 1\}$
- 3) oracle ζ

Result: Valid migration sequence $\mathcal{S}$ on the condensed graph;

```
1 Step 0: Transitively reduce  $E$ ;  
2  $E \leftarrow$  transitive reduction of  $E$ ;  
3 Step 1: Strongly connected components;  
4  $\{C_1, \dots, C_k\} \leftarrow$  strongly connected components of  $E$ ;;  
5 Step 2: Condensation graph;  
6  $V/c \leftarrow \{C_1, \dots, C_k\}$ ;  
7  $E/c \leftarrow \{(C_i, C_j) \mid \exists (u, v) \in E, u \in C_i, v \in C_j, i \neq j\}$ ;  
8  $E/c \leftarrow$  transitive reduction of  $E/c$ ;  
9 Initialize  $visited(C_i) \leftarrow false$  for all  $C_i \in V/c$ ;  
10 Initialize migration sequence  $\mathcal{S} \leftarrow []$ ;  
11 while there exists  $C_i \in V/c$  with  $visited(C_i) = false$  do  
12   Select a cluster  $C_i$  such that  $\forall C_j, (C_i, C_j) \notin E/c$ ;  
13   if  $\zeta(\mathcal{C}[\mathcal{S} \cup C_i]) = 1$  then  
14      $visited(C_i) \leftarrow true$ ;  
15      $\mathcal{S} \leftarrow \mathcal{S} \cup C_i$ ;  
16      $E/c \leftarrow$  transitive reduction of  $E/c$ ;  
17   else  
18     Invisible dependency detection:  
19     Step A: Identify one blocking cluster and add implicit dependency  
20     foreach  $u \in C_i$  do  
21       foreach  $v \in (V \setminus C_i) \setminus \mathcal{S}$  do  
22         if  $P(u, v) = 1$  then  
23            $E \leftarrow E \cup \{(u, v), (v, u)\}$  ;  
24           break;  
25     Step B: Recompute SCCs after fusion  
26      $V/c \leftarrow SCCs(E)$ ;  
27     Step C: Recompute condensation graph  
28      $E/c \leftarrow \emptyset$ ;  
29     foreach  $C_x \in V/c$  do  
30       foreach  $C_y \neq C_x$  do  
31         if  $\exists (a, b) \in E$  with  $a \in C_x$  and  $b \in C_y$  then  
32            $E/c \leftarrow (C_x, C_y)$ ;  
33     Step E: Reset visited flags  
34     foreach  $C_{new} \in V/c$  do  
35        $visited(C_{new}) \leftarrow false$ ;  
36     Step F:  $E/c \leftarrow$  transitive reduction of  $E/c$ ;  
37 return  $(\mathcal{S})$ ;
```

migration graph refinement by helping identify implicit dependencies.

An automotive case study with ECU, TCU and back-end components shows that key hierarchies for secure boot, shared MAC keys and secure-access mechanisms induce constraints that are not captured at CBOM level, but are detected and encoded by our algorithm 1 into joint migration clusters.

Finally, we formalized PQC migration as a combination of a node wise migration function, a family of dependency specific test functions aggregated into a global predicate, and an execution oracle. Within this framework we formally defined the migration related dependency

types, formulated a clustering and detection algorithm and proved that the algorithm terminates with a migration sequence that is valid and complete with respect to the discovered dependencies.

This paper provides three major insights: (1) Key management and security controls drive migration complexity. (2) Cluster-based strategies require explicit treatment of implicit dependencies. (3) CBOMs, graph models, and tooling should form an iteratively refined workflow. Future work integrates risk and cost, relaxes assumptions (e.g. on the oracle), automates test derivation, and scales to larger infrastructures.

References

- [1] Post-Quantum Cryptography Coalition (PQCC) and The MITRE Corporation. *Post-Quantum Cryptography (PQC) Migration Roadmap*. Technical Report 24-03931-7. Approved for public release. Distribution unlimited. ©2025 The MITRE Corporation. The MITRE Corporation, May 2025. URL: <https://pqcc.org/wp-content/uploads/2025/05/PQC-Migration-Roadmap-PQCC-2.pdf>.
- [2] Gorjan Alagic et al. *Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process*. en. 2022-09-29 04:09:00 2022. DOI: <https://doi.org/10.6028/NIST.IR.8413-upd1>.
- [3] Nouri Alnahawi et al. “On the State of Post-Quantum Cryptography Migration”. In: *Gesellschaft für Informatik*, Bonn, 2021, pp. 907–941. ISBN: 978-3-88579-708-1. DOI: 10.18420/informatik2021-078.
- [4] Alessandro Amadori et al. *The PQC Migration Handbook: Guidelines for Migrating to Post-Quantum Cryptography – Revised and Extended Second Edition*. Tech. rep. TNO, Applied Cryptography et al., Dec. 2024. URL: <https://publications.tno.nl/publication/34643386/fXcPVHsX/TNO-2024-pqc-en.pdf>.
- [5] ANSSI. *Avis de l'ANSSI sur la migration vers la cryptographie post-quantique (suivi 2023)*. Tech. rep. ANSSI, 2023.
- [6] AUTOSAR. *Specification of Secure Onboard Communication AUTOSAR CP R23-11*. Ed. by Standard. 2023. URL: https://www.autosar.org/fileadmin/standards/R23-11/CP/AUTOSAR_CP_SWS_SecureOnboardCommunication.pdf.
- [7] Manuel Barbosa et al. *X-Wing: The Hybrid KEM You've Been Looking For*. Cryptology ePrint Archive, Paper 2024/039. 2024. DOI: 10.62056/a3qj89n4e. URL: <https://eprint.iacr.org/2024/039>.
- [8] Carlos Benitez. *Mapping Quantum Threats: An Engineering Inventory of Cryptographic Dependencies*. 2025. arXiv: 2509.24623 [cs.CR]. URL: <https://arxiv.org/abs/2509.24623>.
- [9] Stefan Ehlen et al. *Kryptografie quantensicher gestalten. Grundlagen, Entwicklungen*. Tech. rep. Empfehlungen. Technical report. BSI-Bro21/01, Bundesamt für Sicherheit in der Informationstechnik, 2021.
- [10] ETSI. *CYBER; Quantum-Safe Cryptography (QSC); A Repeatable Framework for Quantum-Safe Migrations*. Tech. rep. TR 104 016. 2024.
- [11] European Commission. *Commission Recommendation (EU) 2024/1101 of 11 April 2024 on a Coordinated Implementation Roadmap for the transition to Post-Quantum Cryptography*. <https://eur-lex.europa.eu/eli/reco/2024/1101/oj>. OJ L 1101, 12.4.2024. Apr. 2024.
- [12] Federal Office for Information Security. *Cryptographic Mechanisms: Recommendations and Key Lengths*. Technical Guideline BSI TR-02102-1. BSI, Jan. 2025.
- [13] Lov K Grover. “A fast quantum mechanical algorithm for database search”. In: *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. 1996, pp. 212–219.
- [14] Khondokar Fida Hasan et al. “A Framework for Migrating to Post-Quantum Cryptography: Security Dependency Analysis and Case Studies”. en. In: *IEEE Access* 12 (2024), pp. 23427–23450. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3360412. (Visited on 09/17/2025).
- [15] Panos Kampanakis and Tancrède Lepoint. “Vision paper: Do we need to change some things? Open questions posed by the upcoming post-quantum migration to existing standards and deployments”. In: *International Conference on Research in Security Standardisation*. Springer. 2023, pp. 78–102.
- [16] Daniel Loebenberger et al. *On the Formalization of Cryptographic Migration*. 2025. arXiv: 2408.05997 [cs.CR]. URL: <https://arxiv.org/abs/2408.05997>.
- [17] Chujiao Ma et al. “CARAF: crypto agility risk assessment framework”. In: *Journal of Cybersecurity* 7.1 (2021), tyab013.
- [18] Georg Macher et al. “A Review of Threat Analysis and Risk Assessment Methods in the Automotive Context”. In: *Computer Safety, Reliability, and Security*. Ed. by Amund Skavhaug, Jérémie Guiochet, and Friedemann Bitsch. Cham: Springer International Publishing, 2016, pp. 130–141. ISBN: 978-3-319-45477-1.
- [19] Justin J. Miller. “Graph Database Applications and Concepts with Neo4j”. In: 2013. URL: <https://api.semanticscholar.org/CorpusID:17523312>.
- [20] Micha Moffie et al. *Cryptoscope: Analyzing cryptographic usages in modern software*. 2025. arXiv: 2503.19531 [cs.CR]. URL: <https://arxiv.org/abs/2503.19531>.
- [21] Christian Näther et al. “Migrating Software Systems Toward Post-Quantum Cryptography-A Systematic Literature Review”. In: *IEEE Access* 12 (2024), pp. 132107–132126. DOI: 10.1109/ACCESS.2024.3450306.
- [22] OWASP Foundation and CycloneDX Feature Working Group on Cryptography. *Authoritative Guide to CBOM*. Technical Report. Version First Edition. Creative Commons Attribution 4.0 International. OWASP Foundation, Apr. 2024. URL: https://cyclonedx.org/guides/OWASP_CycloneDX-Authoritative-Guide-to-CBOM-en.pdf.
- [23] Gina M. Raimondo and Laurie E. Locascio. “FIPS-203: Module-Lattice-Based Key-Encapsulation Mechanism Standard”. In: *Federal Information Processing Standards Publication* 203 (2025), pp. 1–56. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf>.
- [24] Gina M. Raimondo and Laurie E. Locascio. “FIPS-204: Module-Lattice-Based Digital Signature Standard”. In: *Federal Information Processing Standards Publication* 204 (2025), pp. 1–65. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf>.
- [25] Gina M. Raimondo and Laurie E. Locascio. “FIPS-205: Stateless Hash-Based Digital Signature Standard”. In: *Federal Information Processing Standards Publication* 205 (2025), pp. 1–61. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf>.

//nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf.

- [26] Norrathep Rattanavipanon, Jakapan Suaboot, and Warodom Werapun. “A Toolchain for Assisting Migration of Software Executables Towards Post-Quantum Cryptography”. In: *IEEE Access* 13 (2025), pp. 4368–4380. DOI: 10.1109/ACCESS.2024.3524309.
- [27] Nicolai Schmitt et al. “On Criteria and Tooling for Cryptographic Inventories”. In: *Sicherheit 2024*. Bonn: Gesellschaft für Informatik e.V., 2024, pp. 49–63. ISBN: 978-3-88579-739-5. DOI: 10.18420/sicherheit2024_003.
- [28] Peter W Shor. “Algorithms for quantum computation: discrete logarithms and factoring”. In: *Proceedings 35th annual symposium on foundations of computer science*. Ieee. 1994, pp. 124–134.
- [29] Mirosław Staron and Darko Durisic. “AUTOSAR Standard”. In: *Automotive Software Architectures: An Introduction*. Cham: Springer International Publishing, 2017, pp. 81–116. ISBN: 978-3-319-58610-6. DOI: 10.1007/978-3-319-58610-6_4. URL: https://doi.org/10.1007/978-3-319-58610-6_4.
- [30] Meena Singh Dilip Thakur et al. “Enterprise Post Quantum Cryptography Migration Tools”. In: *2024 16th International Conference on COMMunication Systems & NETWORKS (COMSNETS)*. 2024, pp. 327–329. DOI: 10.1109/COMSNETS59351.2024.10427442.

Appendix A. Proofs

A.1. Lemma 5.1

[Termination]

Proof. Let n_0 be the initial number of strongly connected components (SCCs) of the graph $G = (V, E)$ calculated in “Step 1”. At each iteration of the main loop, the algorithm performs one of the following two operations:

- it successfully migrates a cluster C , marks it `visited=true` and adds it to the sequence $\mathcal{S}$ (the number of unvisited SCCs is decreases by 1);
- or the migration fails: the algorithm detects via P a pair (u, v) such that $P(u, v) = 1$, adds the edges (u, v) and (v, u) , and recalculates the SCCs; the reciprocal addition creates a cycle between the two SCCs containing u and v , and at least two distinct SCCs merge, so the number of unvisited SCCs strictly decreases.

The number of SCCs to migrate can only decrease a finite number of times (at most $n_0 - 1$). Between two merges, there must be at least one successful migration (otherwise, detection via P would immediately produce a new merge). Therefore, the main “While” loop cannot be infinite, and the algorithm terminates after a finite number of iterations. $\square$

A.2. Lemma 5.2

[correctness]

Proof. We prove the lemma using a loop invariant in the sense of Hoare logic.

Loop invariant: At the beginning and end of each iteration of the main loop, the currently constructed sequence $\mathcal{S}$ satisfies: $I(\mathcal{S})$: $\mathcal{S}$ is a valid migration sequence.

- **Initialization:** Before entering the loop, the sequence is empty: $\mathcal{S} = \emptyset$. By the consistency of the oracle ζ , we have: $\zeta(\mathcal{C}[\emptyset]) = 1$. Therefore, the invariant is true before the first iteration.
- **Conservation:** Suppose the invariant is true at the beginning of an iteration, that is: $\mathcal{S}$ is a valid migration sequence. Now let C_i be the next non visited cluster. Two cases can occur:

- **The migration of cluster C_i is successful.** The algorithm tests $\zeta(\mathcal{C}[\mathcal{S} \cup C_i])$ and, since it is valid, it adds it to the sequence:

$$\mathcal{S} \leftarrow \mathcal{S} \cup [C_i].$$

We then have, by definition of the validity of a migration sequence that the new $\mathcal{S}$ is also a valid migration sequence. The invariant is therefore preserved.

- **The migration fails.** The algorithm identifies an implicit dependency via P , reconstructs the strongly connected components, and does not modify the sequence $\mathcal{S}$. Thus, the invariant is still preserved.

- **Termination:** Consequently, if the loop terminates, the invariant holds true. Thus, the final sequence $\mathcal{S}$ provided by the algorithm is a valid migration sequence.

Since the loop invariant is initially true, preserved at each iteration and still true at the exit, the correction is established. $\square$

A.3. Lemma 5.3

[Completeness]

Completeness. Let $\mathcal{S}$ be the sequence obtained at the termination of the algorithm. Suppose, for the sake of contradiction, that there exists a node $x \in V$ such that $x \notin \bigcup_{C \in \mathcal{S}} C$. Then x belongs to a final SCC C_x of the partition V/c at the time of termination. But the algorithm only terminates when all clusters have been marked `visited=true`, a condition that is only satisfied if each cluster has been successfully migrated and added to $\mathcal{S}$. Thus, the SCC C_x must be in $\mathcal{S}$, a contradiction. Therefore, no node is left out, and the union of the clusters of $\mathcal{S}$ is indeed V . $\square$

A.4. Theorem 5.4

[Complete correctness of the algorithm]

Proof. The three points are exactly the respective conclusions of Lemma A.1, Lemma A.2 and Lemma A.3 above; their conjunction provides the stated result. $\square$

Appendix B.

Machine-Readable Representation of the Application Example

This section provides a machine-readable representation based on a graph data based on Neo4J [19]. The scripts are written in Cypher⁸ to model the application example introduced in the main text (cf. Fig. 2 and 3) and further described in appendix 3. Supplying the example as an importable graph dataset enables readers to reconstruct the exact graph used in the article and to explore it with example queries (see below) as well as with their own queries, algorithms, and tools. In the following, we briefly describe how to import the provided Cypher script into a Neo4J⁹ database and outline example queries that can be used as a starting point for further explorations of the application example and the method describes in this paper.

The file `setup_graph_db.cypher` (provided as supplementary material) contains the complete set of CREATE statements required to instantiate the graph in a graph database which supports Cypher. Executing the script yields a graph database state that matches the structure of Fig. 3. The script has been imported into and tested with Neo4J version 2025.10.1.¹⁰

B.1. Neo4J Installation and Data Import

This section briefly outlines the steps required to load the example graph into a Neo4J database. For more details and alternative installation options, the official Neo4j documentation¹¹ should be consulted.

- 1) The easiest installation and usage option is provided by installing “Neo4J Desktop”. Follow the installation instructions on <https://neo4j.com/docs/desktop/current/installation/>.
- 2) Open Neo4J Desktop and create a new local database instance. This step requires providing a database name (e.g. `pqc_mig`), a username (default: `neo4j`) and a corresponding password.
- 3) Start the database instance by clicking on the play button symbol.
- 4) Connect to the database instance by clicking on the Connect-pulldown-menu and selecting the menu item “query”.
- 5) Paste the contents of the file `setup_graph_db.cypher` into the query field and click on the play button symbol next to the query field (`neo4j$`).
- 6) This should result in the message “Created 65 nodes, created 190 relationships, set 65 properties, added 65 labels”.

After successful execution, the database contains the full application example as used in this article.

8. <https://neo4j.com/docs/cypher-manual/current/>

9. <https://neo4j.com/product/neo4j-graph-database/>

10. <https://neo4j.com/release-notes/database/neo4j-2025-10-1/>

11. <https://neo4j.com/docs/>

```
MATCH (n) - [r] - (m)
RETURN n, r, m;
```

Listing 1. display all nodes and edges

B.2. APOC Installation

Some of the below-mentioned queries require the Neo4J library APOC to be installed. Follow the instructions on <https://neo4j.com/docs/apoc/current/installation/> for this purpose and restart the database instance.

B.3. Example Queries

The remainder of this appendix presents a set of example Cypher queries that can be executed on the imported graph. These queries illustrate how the dataset can be explored, how the structural properties described in the main text can be inspected, and how the example can serve as a basis for further experiments. For ease of use (e.g., copy-and-paste into the Neo4j Browser), all example queries presented in this appendix are also provided in the accompanying file `example_queries.cypher`.

The example queries can be pasted into the query field and executed by clicking on the play button symbol next to the query field. The corresponding result is displayed below the query field. The result is displayed as a graph if the option “Graph” is enabled at the top of the display pane.

B.3.1. Display All Nodes and Edges. The query in Listing 1 display all contents (nodes, edges) of the database (s. Fig. 4, Fig. 5).

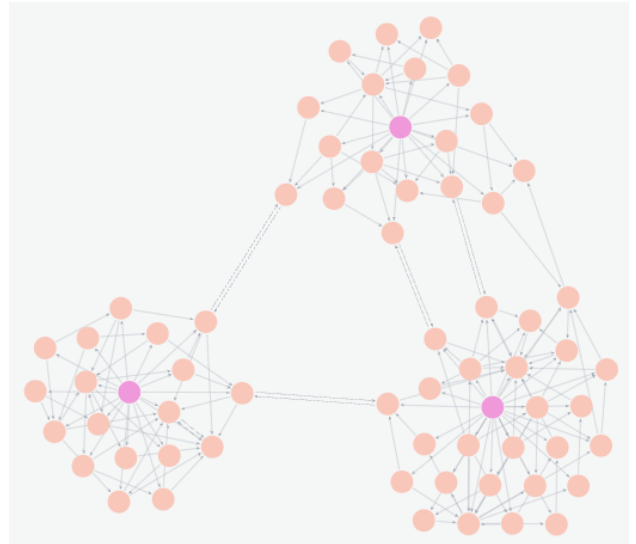


Figure 4. all nodes and edges

B.3.2. Display All Direct Connections to one Node.

The query in Listing 2 displays all connections (ingoing/outgoing) to one specific node (s. Fig. 6). The specific component and node can be filtered by specifying `c.name` and `n.name`. Clicking on one of the displayed neighboring nodes expands the graph further.

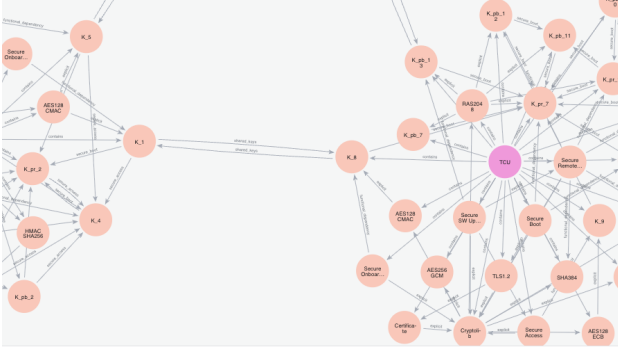


Figure 5. all nodes and edges (zoomed-in)

```

MATCH (c:Container)-[:contains]-(n:Node)
WHERE c.name = 'TCU'
AND n.name = 'Secure SW Update'
MATCH (n:Node)-[r]-(m:Node)
RETURN n, r, m;

```

Listing 2. display direct connections to one node

B.3.3. Display Redundant Explicit Dependencies. The query in Listing 3 displays redundant explicit dependencies, which are redundant in the sense that the graph can be (further) transitively reduced by getting rid of these dependencies (s. Fig. 7).

B.3.4. Display Implicit Dependencies. The query in Listing 4 displays all implicit dependencies which are based on a secure access dependency (s. Fig. 8). The query in Listing 5 displays all implicit dependencies (s. Fig. 9).

B.3.5. Display Cycles. These queries require the APOC library. The query in Listing 6 displays all cycles in the graph (s. Fig. 10). The query in Listing 7 displays all cycles and their relationships to the components that contain them (s. Fig. 11).

B.3.6. Display All Edge Types. The query in Listing 8 displays all edge types in the database (s. Fig. 12).

B.3.7. Clear Database. The query in Listing 9 permanently deletes all database contents.

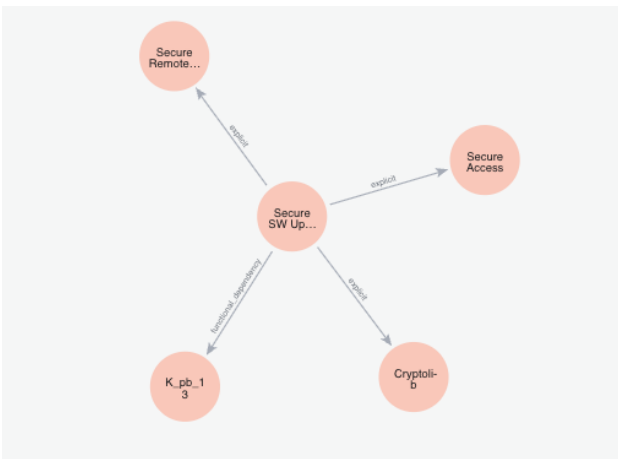


Figure 6. connections to one node

```

MATCH (n:Node)-[r:explicit]->(m:Node)
WHERE EXISTS {
  MATCH path = (n)-[:explicit*2..]->(m)
  WHERE NONE (rel IN relationships(path)
    where rel = r)
}
RETURN n, r, m;

```

Listing 3. display redundant explicit dependencies

```

MATCH (n:Node)-[r:secure_access]->(m:Node)
WHERE NOT EXISTS {
  MATCH (n:Node)-[:explicit*1..]->(m:Node)
}
RETURN n, r, m;

```

Listing 4. display all implicit dependencies based on functional key dependencies

```

MATCH (n:Node)-[r:shared_keys|secure_boot|
  secure_access|communication_dependency|
  functional_dependency]->(m:Node)
WHERE NOT EXISTS {
  MATCH (n:Node)-[:explicit*1..]->(m:Node)
}
RETURN n, r, m;

```

Listing 5. display all implicit dependencies

```

MATCH (n:Node)
WITH collect(n) as nodes
CALL apoc.nodes.cycles(nodes)
YIELD path RETURN path;

```

Listing 6. display all cycles

```

MATCH (n:Node)
WITH collect(n) AS nodes
CALL apoc.nodes.cycles(nodes)
YIELD path
WITH path, nodes(path) AS cycleNodes
MATCH (m:Node)-[r]-(c:Container)
WHERE m IN cycleNodes
RETURN path, m, r, c;

```

Listing 7. display all cycles + dependency between nodes and the three container components (ECU; TCU; Server) (zoomed-in)

```

MATCH ()-[r]->()
RETURN DISTINCT type(r) AS relType
ORDER BY relType;

```

Listing 8. all edge types

```

MATCH (n)
DETACH DELETE n;

```

Listing 9. clear database

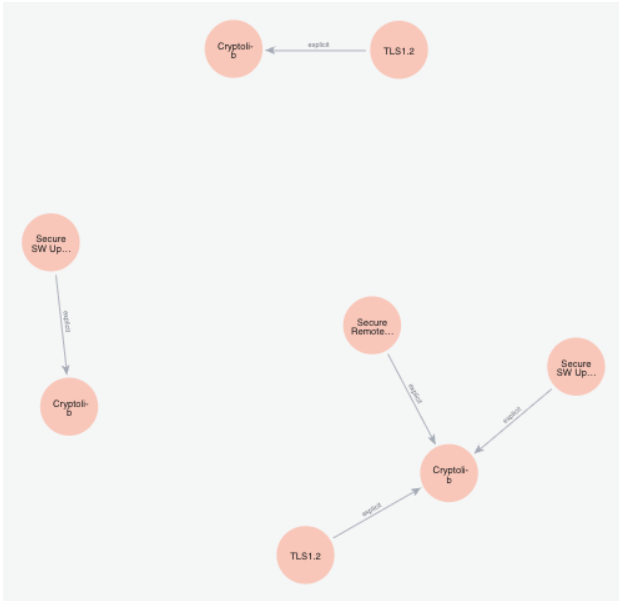


Figure 7. redundant explicit dependencies

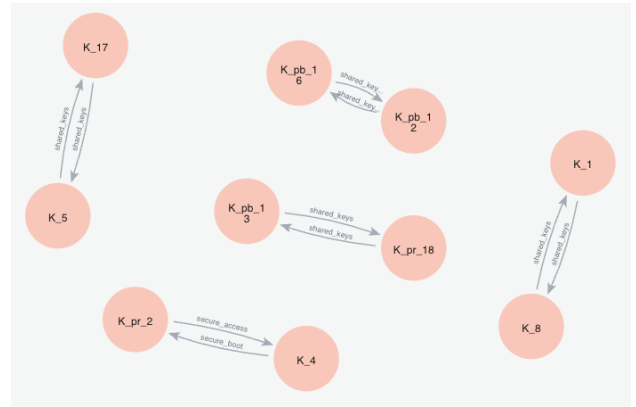


Figure 10. all cycles

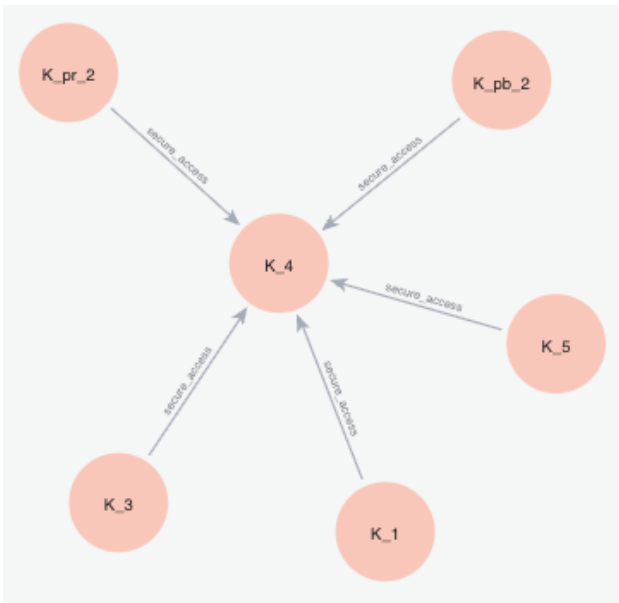


Figure 8. implicit dependencies (secure access)

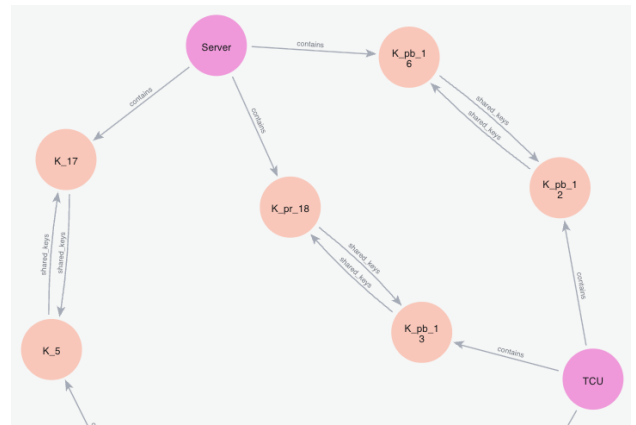


Figure 11. all cycles with containing components

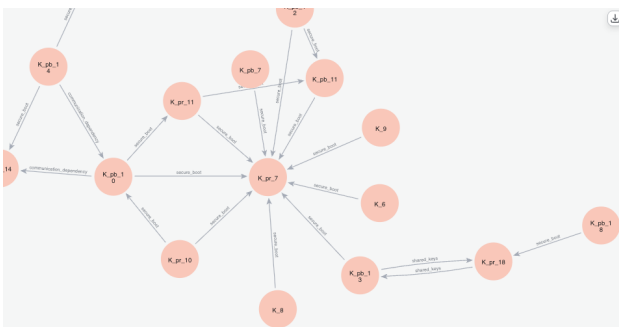


Figure 9. implicit dependencies (zoomed-in)

Table	RAW
relType	
1	"communication_dependency"
2	"contains"
3	"explicit"
4	"functional_dependency"
5	"secure_access"
6	"secure_boot"
7	"shared_keys"

Figure 12. all edge types